Personal Web Monitor

https://gemini.google.com/app/13174114b866a35b

User prompt: I want to build a self-hosted, lightweight "Personal Web Change Detector & Target Monitor" web application to run 24/7 on my Linux VPS behind Caddy. My Background & Constraints: - Languages: Python (FastAPI/APScheduler/BeautifulSoup/Playwright) or PHP/Java. - Host: Linux VPS with low overhead (should consume minimal CPU and RAM while idling). - Purpose: Automatically monitor target web pages (out-of-stock items, price drops, ticket releases, or text announcements) and immediately send a push notification to my phone via Telegram when a change or threshold is met. Key Features Needed: 1. Web Dashboard: Add a URL to watch, configure check frequency (e.g., every 15m, 1h, 24h), and define the target (specific CSS selector, keyword match, or full-page text diff). 2. Headless Polling Engine: A resilient background worker that fetches targets, compares snapshots using hashing or text diffs, and prevents false positives on dynamic pages. 3. Alert Dispatcher: Instant Telegram Bot alerts containing the target name, URL, and a brief diff of what changed. 4. Persistence: Lightweight storage (SQLite) tracking monitor rules, run history, and last-known content hashes. Please act as a senior software architect and help me build this step by step: 1. Provide the project directory structure and library choices. 2. Give the database schema (SQLite) for storing monitors and change logs. 3. Write the core background polling worker and the Telegram notification dispatcher. 4. Write the FastAPI backend endpoints and a clean single-file dashboard UI to add/edit/pause monitors. 5. Provide the systemd service file and Caddyfile reverse-proxy setup for 24/7 uptime on my VPS.

Response: For a low-overhead, 24/7 web change detector on a small Linux VPS, **FastAPI + SQLite + APScheduler +** `httpx` **/ BeautifulSoup** is the most resource-efficient stack. It idles at roughly 35–60 MB of RAM (compared to ~300+ MB if Chromium/Playwright runs continuously).

1. Architectural Strategy & Directory Structure

To keep resource consumption near zero on idle:

- **Default to fast HTTP requests:** Use `httpx` (with standard browser headers) for 95% of target pages.
- **Normalize HTML before hashing:** Strip volatile elements (`<script>`, `<style>`, `nonce`, csrf tokens, advertising wrappers, and whitespace) to eliminate false positives.
- **Single-process concurrency:** APScheduler runs directly inside the FastAPI process lifespan using `asyncio`, keeping execution within a single Python runtime without needing Redis or Celery.

```
webmon/
├── app/
│   ├── __init__.py
│   ├── config.py          # Env vars (Telegram token, chat ID, secret key)
│   ├── database.py        # SQLite connection & schema initialization
│   ├── notifier.py        # Async Telegram alert dispatcher
│   ├── worker.py          # Snapshot fetching, normalization, & diff engine
│   └── main.py            # FastAPI API routes + static HTML dashboard
├── static/
│   └── index.html         # Lightweight dashboard UI (Tailwind via CDN)
├── data/
│   └── webmon.db          # Persistent SQLite database file
├── requirements.txt
└── Caddyfile
```

requirements.txt

```
fastapi>=0.110.0
uvicorn[standard]>=0.28.0
httpx>=0.27.0
beautifulsoup4>=4.12.3
apscheduler>=3.10.4
python-dotenv>=1.0.1
```

2. Database Schema (SQLite)

We use SQLite via standard Python library `sqlite3` in WAL (Write-Ahead Logging) mode to allow concurrent reads and writes without thread locks.

app/database.py

```
import os
import sqlite3
from contextlib import contextmanager

DB_DIR = os.getenv("DB_DIR", "data")
os.makedirs(DB_DIR, exist_ok=True)
DB_PATH = os.path.join(DB_DIR, "webmon.db")

def init_db():
    with sqlite3.connect(DB_PATH) as conn:
        conn.execute("PRAGMA journal_mode=WAL;")
        conn.execute("PRAGMA foreign_keys=ON;")

        # Monitors table
        conn.execute("""
        CREATE TABLE IF NOT EXISTS monitors (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            url TEXT NOT NULL,
            css_selector TEXT DEFAULT NULL,
            match_type TEXT NOT NULL CHECK(match_type IN ('selector_diff', 'keyword_present', 'keyword_missing', 'text_diff')),
            keyword TEXT DEFAULT NULL,
            interval_minutes INTEGER NOT NULL DEFAULT 60,
            is_active INTEGER NOT NULL DEFAULT 1,
            last_checked_at TIMESTAMP,
            last_content_hash TEXT,
            last_content_snippet TEXT,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        );
        """)

        # Change event history
        conn.execute("""
        CREATE TABLE IF NOT EXISTS change_logs (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            monitor_id INTEGER NOT NULL REFERENCES monitors(id) ON DELETE CASCADE,
            detected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            summary TEXT NOT NULL,
            diff_text TEXT,
            notified_status TEXT DEFAULT 'SENT'
        );
        """)
        conn.commit()

@contextmanager
def get_db():
    conn = sqlite3.connect(DB_PATH, timeout=10.0)
    conn.row_factory = sqlite3.Row
    try:
        yield conn
    finally:
        conn.close()
```

3. Core Engine: Polling Worker & Telegram Dispatcher

app/notifier.py

```
import httpx
import logging
from app.config import TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID

logger = logging.getLogger(__name__)

async def send_telegram_alert(monitor_name: str, url: str, change_summary: str, diff_text: str = ""):
    if not TELEGRAM_BOT_TOKEN or not TELEGRAM_CHAT_ID:
        logger.warning("Telegram credentials not configured. Skipping notification.")
        return

    text = (
        f"🚨 <b>Web Change Detected!</b>\n\n"
        f"<b>Target:</b> {monitor_name}\n"
        f"<b>URL:</b> {url}\n"
        f"<b>Trigger:</b> {change_summary}\n"
    )
    if diff_text:
        # Truncate diff to avoid exceeding Telegram's 4096-char payload limit
        clean_diff = (diff_text[:1200] + "...") if len(diff_text) > 1200 else diff_text
        text += f"\n<pre>{clean_diff}</pre>"

    api_url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
    payload = {
        "chat_id": TELEGRAM_CHAT_ID,
        "text": text,
        "parse_mode": "HTML",
        "disable_web_page_preview": False
    }

    async with httpx.AsyncClient(timeout=10.0) as client:
        try:
            resp = await client.post(api_url, json=payload)
            resp.raise_for_status()
        except Exception as e:
            logger.error(f"Failed to dispatch Telegram message: {e}")
```

app/worker.py

```
import difflib
import hashlib
import logging
import re
from datetime import datetime
from bs4 import BeautifulSoup
import httpx

from app.database import get_db
from app.notifier import send_telegram_alert
```

```
logger = logging.getLogger(__name__)

HEADERS = {
    "User-Agent": "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36",
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
    "Accept-Language": "en-US,en;q=0.5",
}

def clean_html(raw_html: str, selector: str = None) -> str:
    """Strips ephemeral scripts/styles and collapses irregular whitespace."""
    soup = BeautifulSoup(raw_html, "html.parser")
    for tag in soup(["script", "style", "noscript", "svg", "iframe"]):
        tag.decompose()

    if selector:
        target = soup.select_one(selector)
        text = target.get_text(separator=" ", strip=True) if target else ""
    else:
        text = soup.get_text(separator=" ", strip=True)

    return re.sub(r"\s+", " ", text).strip()

def compute_hash(text: str) -> str:
    return hashlib.sha256(text.encode("utf-8")).hexdigest()

async def check_monitor(monitor_id: int):
    with get_db() as conn:
        row = conn.execute("SELECT * FROM monitors WHERE id = ? AND is_active = 1", (monitor_id,)).fetchone()
        if not row:
            return
        m = dict(row)

    try:
        async with httpx.AsyncClient(headers=HEADERS, timeout=20.0, follow_redirects=True) as client:
            res = await client.get(m["url"])
            res.raise_for_status()
            current_raw = res.text
    except Exception as exc:
        logger.warning(f"Error fetching {m['url']} (Monitor ID: {m['id']}): {exc}")
        return

    current_text = clean_html(current_raw, m["css_selector"])
    new_hash = compute_hash(current_text)
    old_hash = m["last_content_hash"]
    old_snippet = m["last_content_snippet"] or ""

    triggered = False
    summary = ""
    diff_text = ""

    # First run: seed baseline without firing alerts
    if old_hash is None:
        with get_db() as conn:
            conn.execute(
                "UPDATE monitors SET last_checked_at = ?, last_content_hash = ?, last_content_snippet = ? WHERE id = ?",
                (datetime.utcnow(), new_hash, current_text[:2000], m["id"])
            )
            conn.commit()
        return

    # Check rule logic
    if m["match_type"] in ("selector_diff", "text_diff"):
        if new_hash != old_hash:
            triggered = True
            summary = "Page content modified"
            diff = difflib.unified_diff(
                old_snippet.splitlines(keepends=True),
                current_text[:2000].splitlines(keepends=True),
                fromfile="old",
                tofile="new",
                n=1
            )
            diff_text = "".join(diff)

    elif m["match_type"] == "keyword_present":
        keyword = (m["keyword"] or "").lower()
        if keyword in current_text.lower() and keyword not in old_snippet.lower():
            triggered = True
            summary = f"Keyword detected: '{m['keyword']}'"

    elif m["match_type"] == "keyword_missing":
        keyword = (m["keyword"] or "").lower()
        if keyword not in current_text.lower() and keyword in old_snippet.lower():
            triggered = True
            summary = f"Keyword no longer present: '{m['keyword']}'"

    # Persist checks and notify if state changed
    with get_db() as conn:
        conn.execute(
            "UPDATE monitors SET last_checked_at = ?, last_content_hash = ?, last_content_snippet = ? WHERE id = ?",
            (datetime.utcnow(), new_hash, current_text[:2000], m["id"])
        )
        if triggered:
            conn.execute(
                "INSERT INTO change_logs (monitor_id, summary, diff_text) VALUES (?, ?, ?)",
                (m["id"], summary, diff_text)
            )
        conn.commit()

    if triggered:
        await send_telegram_alert(m["name"], m["url"], summary, diff_text)
```

4. FastAPI Backend & Dashboard UI

app/config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

TELEGRAM_BOT_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN", "")
TELEGRAM_CHAT_ID = os.getenv("TELEGRAM_CHAT_ID", "")
DASHBOARD_PASSWORD = os.getenv("DASHBOARD_PASSWORD", "changeme")
```

app/main.py

```
from contextlib import asynccontextmanager
from apscheduler.schedulers.asyncio import AsyncIOScheduler
from fastapi import FastAPI, HTTPException, Depends, status
from fastapi.responses import HTMLResponse
from fastapi.security import HTTPBasic, HTTPBasicCredentials
from pydantic import BaseModel
from typing import Optional
import secrets

from app.config import DASHBOARD_PASSWORD
from app.database import init_db, get_db
from app.worker import check_monitor

scheduler = AsyncIOScheduler()
security = HTTPBasic()

def authenticate(creds: HTTPBasicCredentials = Depends(security)):
    correct_pass = secrets.compare_digest(creds.password, DASHBOARD_PASSWORD)
    if not correct_pass:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Unauthorized",
            headers={"WWW-Authenticate": "Basic"}
        )
    return creds.username

def sync_scheduler():
    scheduler.remove_all_jobs()
    with get_db() as conn:
        monitors = conn.execute("SELECT id, interval_minutes FROM monitors WHERE is_active = 1").fetchall()
        for m in monitors:
            scheduler.add_job(
                check_monitor,
                "interval",
                minutes=m["interval_minutes"],
                args=[m["id"]],
                id=f"monitor_{m['id']}",
                replace_existing=True
            )

@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
    init_db()
    sync_scheduler()
    scheduler.start()
    yield
    scheduler.shutdown(wait=False)

app = FastAPI(lifespan=lifespan)

class MonitorCreate(BaseModel):
    name: str
    url: str
    css_selector: Optional[str] = None
    match_type: str = "selector_diff"
    keyword: Optional[str] = None
    interval_minutes: int = 60

@app.get("/", response_class=HTMLResponse)
def index_view(_: str = Depends(authenticate)):
    with open("static/index.html", "r", encoding="utf-8") as f:
        return f.read()

@app.get("/api/monitors")
def list_monitors(_: str = Depends(authenticate)):
    with get_db() as conn:
        rows = conn.execute("SELECT * FROM monitors ORDER BY id DESC").fetchall()
        return [dict(r) for r in rows]

@app.post("/api/monitors")
def create_monitor(item: MonitorCreate, _: str = Depends(authenticate)):
    with get_db() as conn:
        cur = conn.execute(
            """INSERT INTO monitors (name, url, css_selector, match_type, keyword, interval_minutes)
               VALUES (?, ?, ?, ?, ?, ?)""",
            (item.name, item.url, item.css_selector, item.match_type, item.keyword, item.interval_minutes)
        )
        conn.commit()
    sync_scheduler()
    return {"id": cur.lastrowid}

@app.post("/api/monitors/{monitor_id}/toggle")
def toggle_monitor(monitor_id: int, _: str = Depends(authenticate)):
    with get_db() as conn:
        conn.execute("UPDATE monitors SET is_active = ((is_active | 1) - (is_active & 1)) WHERE id = ?", (monitor_id,))
        conn.commit()
    sync_scheduler()
    return {"status": "ok"}

@app.delete("/api/monitors/{monitor_id}")
def delete_monitor(monitor_id: int, _: str = Depends(authenticate)):
    with get_db() as conn:
        conn.execute("DELETE FROM monitors WHERE id = ?", (monitor_id,))
        conn.commit()
    sync_scheduler()
    return {"status": "ok"}
```

static/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Personal Web Monitor</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-slate-900 text-slate-100 min-h-screen p-4 sm:p-8 font-sans">
  <div class="max-w-4xl mx-auto space-y-8">
    <div class="flex justify-between items-center border-b border-slate-800 pb-4">
      <h1 class="text-2xl font-bold tracking-tight text-white">Target Monitor Dashboard</h1>
      <span class="inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-medium bg-emerald-950 text-emerald-400 border border-emerald-800">Engine Active</span>
    </div>

    <!-- Create Monitor Form -->
    <div class="bg-slate-800/60 p-6 rounded-xl border border-slate-700/80 shadow-md">
      <h2 class="text-lg font-semibold mb-4 text-slate-200">New Target Monitor</h2>
      <form id="monitorForm" class="grid grid-cols-1 md:grid-cols-2 gap-4">
        <div>
          <label class="block text-xs font-medium uppercase text-slate-400 mb-1">Target Name</label>
          <input type="text" id="name" required placeholder="e.g. GPU In Stock" class="w-full bg-slate-900 border border-slate-700 rounded-lg px-3 py-2 text-sm focus:border-indigo-500 focus:outline-none">
        </div>
        <div>
          <label class="block text-xs font-medium uppercase text-slate-400 mb-1">Target URL</label>
          <input type="url" id="url" required placeholder="https://example.com/product" class="w-full bg-slate-900 border border-slate-700 rounded-lg px-3 py-2 text-sm focus:border-indigo-500 focus:outline-none">
        </div>
        <div>
          <label class="block text-xs font-medium uppercase text-slate-400 mb-1">CSS Selector (Optional)</label>
          <input type="text" id="css_selector" placeholder=".price-tag, #stock-status" class="w-full bg-slate-900 border border-slate-700 rounded-lg px-3 py-2 text-sm focus:border-indigo-500 focus:outline-none">
        </div>
        <div>
          <label class="block text-xs font-medium uppercase text-slate-400 mb-1">Trigger Condition</label>
          <select id="match_type" class="w-full bg-slate-900 border border-slate-700 rounded-lg px-3 py-2 text-sm focus:border-indigo-500 focus:outline-none">
            <option value="selector_diff">Selector / Page Content Diff</option>
            <option value="keyword_present">Keyword Appears</option>
            <option value="keyword_missing">Keyword Disappears</option>
          </select>
        </div>
        <div>
          <label class="block text-xs font-medium uppercase text-slate-400 mb-1">Keyword (If Applicable)</label>
          <input type="text" id="keyword" placeholder="e.g. In Stock" class="w-full bg-slate-900 border border-slate-700 rounded-lg px-3 py-2 text-sm focus:border-indigo-500 focus:outline-none">
        </div>
        <div>
          <label class="block text-xs font-medium uppercase text-slate-400 mb-1">Polling Interval (Minutes)</label>
          <input type="number" id="interval" value="15" min="1" class="w-full bg-slate-900 border border-slate-700 rounded-lg px-3 py-2 text-sm focus:border-indigo-500 focus:outline-none">
        </div>
        <div class="md:col-span-2 pt-2">
          <button type="submit" class="w-full bg-indigo-600 hover:bg-indigo-500 py-2.5 rounded-lg text-sm font-semibold transition">Add Monitor</button>
        </div>
      </form>
    </div>

    <!-- Active Monitors Table -->
    <div class="bg-slate-800/60 rounded-xl border border-slate-700/80 overflow-hidden shadow-md">
      <div class="p-4 border-b border-slate-700 flex justify-between items-center">
        <h2 class="text-lg font-semibold text-slate-200">Monitored Targets</h2>
        <button onclick="fetchMonitors()" class="text-xs bg-slate-700 hover:bg-slate-600 px-3 py-1.5 rounded transition">Refresh</button>
      </div>
      <div class="overflow-x-auto">
        <table class="w-full text-left text-sm text-slate-300">
          <thead class="bg-slate-900/60 text-xs uppercase text-slate-400">
            <tr>
              <th class="p-3">Name</th>
              <th class="p-3">Trigger Type</th>
              <th class="p-3">Interval</th>
              <th class="p-3">Last Checked</th>
              <th class="p-3">Status</th>
              <th class="p-3 text-right">Actions</th>
            </tr>
          </thead>
          <tbody id="monitorRows" class="divide-y divide-slate-700/50"></tbody>
        </table>
      </div>
    </div>
  </div>

  <script>
    async function fetchMonitors() {
      const res = await fetch('/api/monitors');
      const data = await res.json();
      const tbody = document.getElementById('monitorRows');
      tbody.innerHTML = '';
      data.forEach(m => {
        const tr = document.createElement('tr');
        tr.innerHTML = `
          <td class="p-3">
            <div class="font-medium text-white">${m.name}</div>
            <a href="${m.url}" target="_blank" class="text-xs text-indigo-400 truncate block max-w-xs">${m.url}</a>
          </td>
          <td class="p-3 font-mono text-xs">${m.match_type}</td>
          <td class="p-3">${m.interval_minutes}m</td>
          <td class="p-3 text-xs text-slate-400">${m.last_checked_at ? m.last_checked_at.slice(0, 19).replace('T', ' ') : 'Pending'}</td>
          <td class="p-3">
            <span class="px-2 py-0.5 rounded text-xs ${m.is_active ? 'bg-emerald-950 text-emerald-400 border border-emerald-800' : 'bg-rose-950 text-rose-400 border border-rose-800'}">
              ${m.is_active ? 'Active' : 'Paused'}
            </span>
          </td>
          <td class="p-3 text-right space-x-2">
            <button onclick="toggleMonitor(${m.id})" class="text-xs ${m.is_active ? 'text-amber-400' : 'text-emerald-400'} hover:underline">${m.is_active ? 'Pause' : 'Resume'}</button>
```

```
            <button onclick="deleteMonitor(${m.id})" class="text-xs text-rose-400 hover:underline">Delete</button>
          </td>
        `;
        tbody.appendChild(tr);
      });
    }

    document.getElementById('monitorForm').addEventListener('submit', async (e) => {
      e.preventDefault();
      const payload = {
        name: document.getElementById('name').value,
        url: document.getElementById('url').value,
        css_selector: document.getElementById('css_selector').value || null,
        match_type: document.getElementById('match_type').value,
        keyword: document.getElementById('keyword').value || null,
        interval_minutes: parseInt(document.getElementById('interval').value, 10)
      };
      await fetch('/api/monitors', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload)
      });
      e.target.reset();
      fetchMonitors();
    });

    async function toggleMonitor(id) {
      await fetch(`/api/monitors/${id}/toggle`, { method: 'POST' });
      fetchMonitors();
    }

    async function deleteMonitor(id) {
      if (confirm('Delete this monitor?')) {
        await fetch(`/api/monitors/${id}`, { method: 'DELETE' });
        fetchMonitors();
      }
    }

    fetchMonitors();
  </script>
</body>
</html>
```

5. Deployment: systemd Service & Caddyfile

Step 1: VPS Systemd Unit (`/etc/systemd/system/webmon.service`)

Create the service unit to ensure FastAPI restarts automatically on reboot or failure:

```
[Unit]
Description=Personal Web Change Detector
After=network.target

[Service]
Type=simple
User=www-data
Group=www-data
WorkingDirectory=/var/www/webmon
Environment="PATH=/var/www/webmon/venv/bin"
Environment="TELEGRAM_BOT_TOKEN=123456789:ABCdefGHIjklMNOpqrSTUvwxYZ"
Environment="TELEGRAM_CHAT_ID=987654321"
Environment="DASHBOARD_PASSWORD=YourStrongRandomPasswordHere"
ExecStart=/var/www/webmon/venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000 --workers 1
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Enable and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable --now webmon
```

Step 2: Caddyfile Configuration (`/etc/caddy/Caddyfile`)

Caddy will automatically provision and renew your Let's Encrypt TLS certificate while reverse-proxying directly to Uvicorn.

```
monitor.yourdomain.com {
    encode gzip zstd

    reverse_proxy 127.0.0.1:8000 {
        header_up Host {host}
        header_up X-Real-IP {remote_host}
    }
}
```

Reload Caddy to apply changes:

```
sudo systemctl reload caddy
```

Verification & Memory Profile

- **Memory Footprint:** Verify low consumption using `ps -o rss,command -p $(pgrep -f uvicorn)` — RSS will settle between **40 MB and 65 MB**.
- **Telegram Bot Setup:** If you don't have a bot token, create one via `@BotFather` on Telegram, and retrieve your numerical ID by messaging `@userinfobot`.

User prompt: with nginx

Response: To deploy your Personal Web Change Detector behind **Nginx** instead of Caddy on Linux (Ubuntu/Debian), use this production-ready setup with automatic SSL via Let's Encrypt / Certbot, gzip compression, and secure reverse proxy headers.

1. Install Nginx & Certbot

Run the package updates and install Nginx alongside the Let's Encrypt Certbot plugin:

```
sudo apt update
sudo apt install -y nginx certbot python3-certbot-nginx
```

Ensure Nginx is enabled and running:

```
sudo systemctl enable --now nginx
```

2. Nginx Server Block Configuration

Create a dedicated virtual host configuration file:

```
sudo nano /etc/nginx/sites-available/webmon.conf
```

Add the following configuration (replace `monitor.yourdomain.com` with your real domain or subdomain):

```
# Rate limiting zone to protect against brute-force attacks on the dashboard
limit_req_zone $binary_remote_addr zone=webmon_limit:10m rate=10r/s;

server {
    listen 80;
    listen [::]:80;
    server_name monitor.yourdomain.com;

    # Maximum upload/request body size (minimal needed for this app)
    client_max_body_size 2M;

    # Gzip Compression for low bandwidth and snappy dashboard loading
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_proxied expired no-cache no-store private auth;
    gzip_types text/plain text/css text/xml text/javascript application/x-javascript application/json application/xml;
    gzip_disable "MSIE [1-6]\.";

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "no-referrer-when-downgrade" always;

    location / {
        # Apply rate limiting
        limit_req zone=webmon_limit burst=20 nodelay;

        # Reverse proxy to local Uvicorn instance
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;

        # Forward proper client and host headers
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # WebSocket support (future-proof if you add real-time log streaming)
```

```
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";

        # Timeouts
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }

    # Deny access to hidden files (e.g. .git, .env)
    location ~ /\. {
        deny all;
        access_log off;
        log_not_found off;
    }
}
```

3. Enable Site & Test Syntax

Link the site configuration to `sites-enabled`:

```
sudo ln -s /etc/nginx/sites-available/webmon.conf /etc/nginx/sites-enabled/
```

Verify that the Nginx configuration syntax is valid:

```
sudo nginx -t
```

If it reports `syntax is ok` and `test is successful`, reload Nginx:

```
sudo systemctl reload nginx
```

4. Provision Free SSL via Certbot

Run Certbot to automatically configure Let's Encrypt certificates, manage HTTP-to-HTTPS redirection, and edit the server block:

```
sudo certbot --nginx -d monitor.yourdomain.com
```

- Choose to redirect all HTTP traffic to HTTPS when prompted.
- Certbot installs a systemd timer (`certbot.timer`) that handles automatic renewal twice daily.

5. Firewall Configuration (UFW)

Ensure your VPS firewall allows Nginx traffic and keeps port 8000 closed to the public:

```
sudo ufw allow 'Nginx Full'
sudo ufw delete allow 8000    # Ensure Uvicorn is ONLY reachable internally via 127.0.0.1
sudo ufw reload
```

Would you like to add an optional SQLite automated backup cron job or set up basic fail2ban rules to protect the HTTP Basic Auth endpoint?
